PRACTICAL REPORT 05

Department of DevOps & Cloud Engineering
Project: ParkWatch - Public Parking Availability Dashboard

Student: Sajiya Shaikh | **Roll No:** 24102A2005
Academic Year: 2026 | **Milestone:** Task 05

Task 05 Report: RESTful API Implementation & Serialization

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student: Sajiya Shaikh

Roll Number: 24102A2005

Milestone: Task 05

Status: COMPLETED

1. Objective

Design and implement a robust, versioned RESTful API under `/api/v1` supporting location querying, real-time event recording, alert monitoring, and consistent RFC-7807 global exception handling.

2. Tools and Technologies Used

- **Framework:** Spring MVC (`@RestController` , `@RequestMapping` , `@PathVariable` , `@RequestParam` , `@RequestBody`)
- **Data Serialization:** Jackson JSON (`ObjectMapper`)
- **Validation:** Jakarta Bean Validation (`@Valid` , `@NotNull` , `@Min`)
- **Exception Handling:** Spring `@RestControllerAdvice`
- **Testing:** Spring `MockMvc` test framework

3. Work Performed

1. Implemented `ParkingApiController` with the following contract:

- `GET /api/v1/summary` : Returns system-wide totals (locations, capacity, occupied, available, occupancy rate).
- `GET /api/v1/locations` : Returns all locations with optional filter params (`district` , `search` , `minAvailable`).
- `GET /api/v1/locations/{id}` : Returns details of a specific parking facility.
- `POST /api/v1/locations` : Registers a new parking facility.
- `PUT /api/v1/locations/{id}` : Updates an existing parking facility.
- `DELETE /api/v1/locations/{id}` : Deletes a facility (HTTP 204).

- `POST /api/v1/locations/{id}/events`: Submits a check-in or check-out event with payload validation.
- `GET /api/v1/locations/{id}/events`: Retrieves event history for a facility.
- `GET /api/v1/events`: Retrieves recent system-wide events.
- `GET /api/v1/alerts`: Returns active capacity alerts (>= 85%).

1. Implemented `GlobalExceptionHandler` returning consistent JSON error payloads containing `timestamp`, `status`, `error`, `message`, and `path`.
2. Added custom exception classes: `ResourceNotFoundException` (HTTP 404), `InvalidOperationException` (HTTP 400).

---

4. Files Created or Modified

- `src/main/java/com/parking/dashboard/controller/ParkingApiController.java`: REST controller.
- `src/main/java/com/parking/dashboard/exception/GlobalExceptionHandler.java`: Central error advice.
- `src/main/java/com/parking/dashboard/exception/ResourceNotFoundException.java`: 404 exception.
- `src/main/java/com/parking/dashboard/exception/InvalidOperationException.java`: 400 exception.
- `src/main/java/com/parking/dashboard/dto/*`: Serialization transfer objects.
- `src/test/java/com/parking/dashboard/controller/ParkingApiControllerTest.java`: 11 MockMvc integration tests.

---

5. Commands Executed

```
# Execute the REST API controller test suite
.\mvnw.cmd test -Dtest=ParkingApiControllerTest
```

Execution Log / Terminal Output

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.parking.dashboard.controller.ParkingApiControllerTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.421 s -- in com.parking.dashboard.contr
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

---

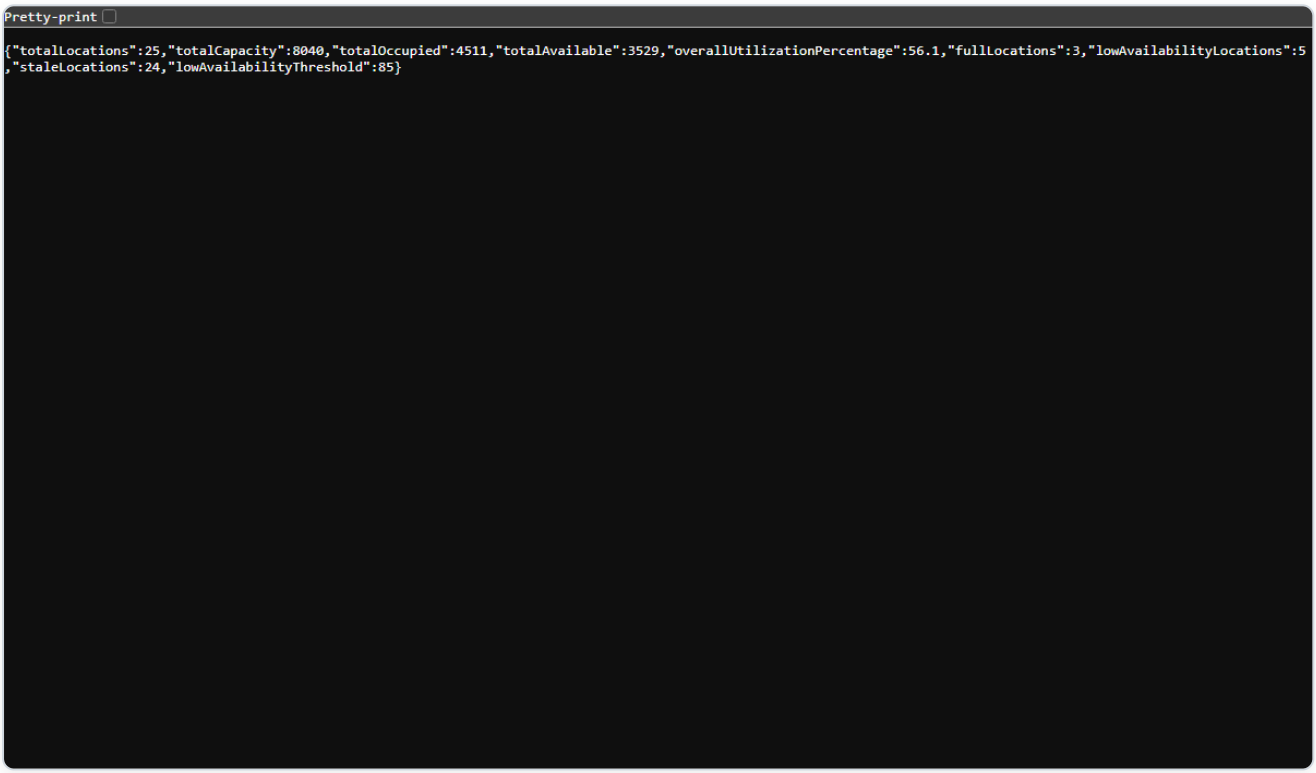
6. Actual Test Results & Verification Evidence

- **11 MockMvc tests passed with 100% success**:
- `GET /api/v1/summary`: Asserts status 200 and JSON path `totalLocations`, `totalCapacity`.
- `GET /api/v1/locations`: Asserts array return and filter functionality.
- `GET /api/v1/locations/{id}`: Asserts 200 for valid ID and 404 for invalid ID.
- `POST /api/v1/locations/{id}/events`: Asserts 200 on valid check-in and 400 on over-capacity check-in.
- Input validation tests: Asserts 400 Bad Request on invalid payloads.

---

7. Screenshots as Evidence

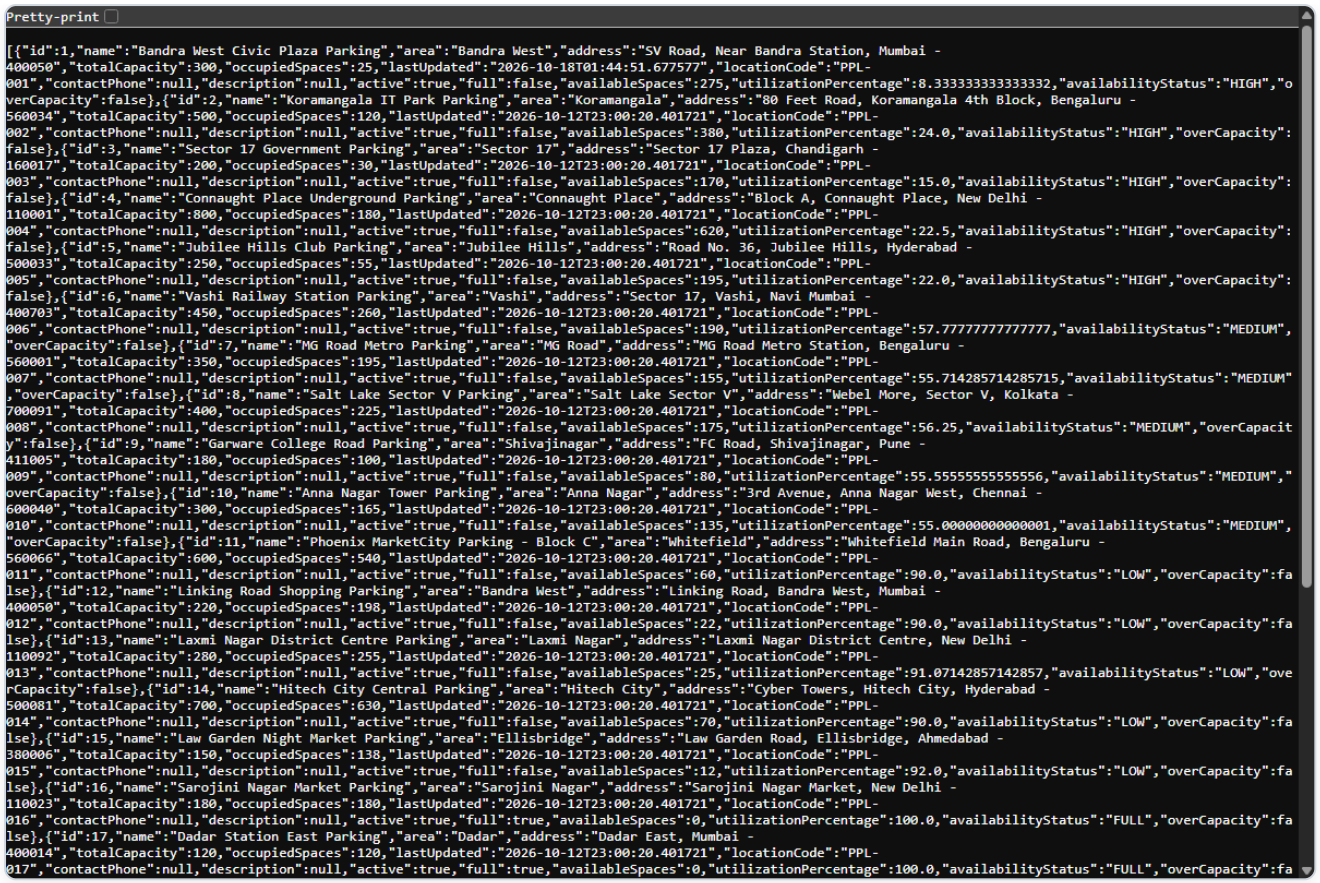
- **REST API Summary Endpoint (`/api/v1/dashboard/summary`):**



API Summary Endpoint

Evidence Description: Live browser query to `/api/v1/dashboard/summary` displaying JSON aggregates: 25 total locations, 8,040 total capacity, 4,511 occupied spaces, and 56.1% overall utilization.

- **REST API Locations List (`/api/v1/locations`):**



API Locations List

Evidence Description: Live browser query returning formatted JSON array of all 25 municipal parking locations with calculated availability statuses.

> **Screenshot Capture Instructions for Student:**

> Start the application and query `http://localhost:8080/api/v1/summary` in Edge or Postman, capture the formatted JSON response (`Win + Shift + S`), and save it to:

> `C:\Users\Admin\Desktop\Public-Parking-DevOps\REPORTS\screenshots\task_05\api_endpoints_postman.png`

---

8. Problems Encountered and Solutions

- **Issue:** Default Spring Boot error handler returning raw stack traces and HTML error pages.
- **Solution:** Built `GlobalExceptionHandler` handling `MethodArgumentNotValidException`, `ResourceNotFoundException`, and `InvalidOperationException` with structured JSON responses.

---

9. Final Status & Remaining Work

- **Status: COMPLETED**
- **Remaining Work:** None. API layer is fully tested and verified.